

—— 多智能体思辨训练系统

AI 辩论场

一套把正规 4v4 辩论赛制、LangGraph workflow、RAG 引用校验与人机协同整合起来的训练系统。 它不是让 AI 替学生辩论，而是让学生在可信、可复盘、可联机的环境里练习论证、反驳和表达。

输入辩题 设置持方、赛制、资料与参与模式	组建辩队 正反各四辩与裁判智能体协同推进	workflow运行 检索、判断、反思、生成、核查、评分	实时对抗 WebSocket 流式同步，人机与联机模式共用	赛后复盘 裁判报告、逐字稿导出与回放分享
--	--	---	---	--

—— 为什么需要它

辩论训练最难的，不是写稿，而是反复经历完整对抗。

学生需要对手、裁判、资料、赛程和反馈同时在线；而现实中，社团排练依赖人力协调，普通 AI 聊天又容易给出没有来源的“漂亮话”。训练一旦缺少可信事实和结构化复盘，就会变成临场背稿。

五个被系统直接回应的痛点

- 缺陪练：很难随时凑齐正反双方和裁判。
- 缺流程：自由聊天无法模拟立论、攻辩、自由辩和总结的压力。
- 缺可信资料：大模型可能编造引用、数据和法规。
- 缺复盘：一次练习之后很难沉淀逐字稿、失误和战场变化。
- 缺部署条件：学校机房、比赛现场常常不能安装复杂环境。

项目的核心判断是：把 AI 辩论做成“可验证的训练场”，比做一个会说话的聊天机器人更有价值。

—— 项目已经做成什么

AI 辩论场把“辩题输入”到“裁判报告”的全链路跑通。

系统支持 AI 自主辩论、用户加入正方、用户加入反方和多人联机模式。创建房间后， 后端按正式赛程推进， 前端实时呈现三栏辩论室、发言流、角色舞台、进度条和工作流树。

每次发言并不是一次普通 LLM 调用，而是经过 RAG 检索、策略规划、方向判断、反思生成、 事实核查、发布和裁判评分的回合循环。赛后可以导出 Markdown/PDF 逐字稿，并通过只读回放分享。

9

正反各四辩 + 紫苑裁判，
形成完整辩论角色阵容。

40

前端展示 workflow 节点，覆盖准备、攻防、总结与裁决。

9

LangGraph 运行时节点，构成单回合智能体内核。

4

AI 自主、人机正方、人机反方、联机多人四类使用场景。

—— 使用路径

从首页建房开始，系统把复杂辩论拆成可执行的训练流程。



这条路径的关键是：学生不只看到 AI 的答案，还能看到“答案是如何被检索、判断、核查并纳入赛程”的过程。

—— 系统设计

架构不是为展示而画，而是支撑多人、实时、可复盘的辩论现场。

表现层

React + Vite 构建首页、辩论室、联机大厅、回放页和管理页。同一套 Web UI 可以在电脑浏览器、手机浏览器和 Electron 桌面壳中运行。

编排层

FastAPI 提供 REST 与 WebSocket，LangGraph 负责单回合智能体循环，YAML 赛程状态机负责正式 4v4 辩论的阶段推进。

能力层

DeepSeek 作为默认 LLM 网关，RAG 与 SQLite 向量库提供资料检索，MongoDB/Redis 用于持久化与实时缓存，也支持内存降级便于现场演示。

技术选型的现实取向很明确：比赛和学校环境未必稳定，所以系统同时保留“完整工程模式”和“先跑起来的降级模式”。

—— 多智能体协同

系统把“一个 AI 发言”拆成一支辩队的协作。

正方辩队

云汐负责开篇立论，澜汐承担驳论与对辩，珂绫负责盘问与小结，青萝收束全场并完成总结陈词。角色职责来自真实 4v4 辩论分工，不是随机的对话人设。

紫苑裁判

裁判不只宣布结果，还参与任务合理性、论点强度、事实风险、总结质量和最终胜负判断。它让辩论过程具备可解释的秩序。

反方辩队

橙律建立反方框架，星白拆解对方论证，凇Z推进盘问，浅笑完成质询与价值收束。双方队内讨论对对手不可见，用信息隔离模拟真实赛场。

创新不在“角色名字很多”，而在于角色职责、赛程阶段、可见性边界和裁判判断被放进同一套状态系统。

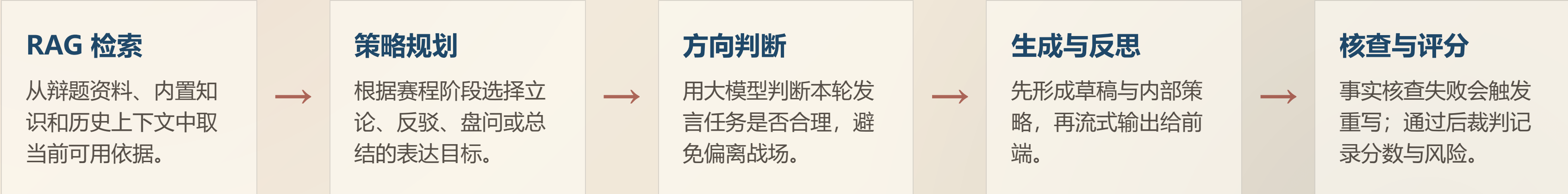
—— 核心工作流

前端看见 40 个节点，后端真正执行 9 个节点；两层共同解释系统如何思考。

展示层：40 节点 用工作流树把赛前准备、立论前准备、驳论检索、自由辩策略、总结陈词和终局裁决展开，让观众看到“当前在哪一步”。	执行层：LangGraph 9 节点 每个 AI 回合走 RAG 检索、策略规划、方向判断、反思、发言生成、事实核查、发布消息、裁判评分和回合路由。	赛程层：正式 4v4 YAML 模板定义立论、攻辩、盘问、自由辩、总结与裁判终局。它让系统按真实辩论节奏推进，而不是让 AI 自由聊天。
---	---	---

—— 运行时内核

一段发言要经过检索、判断、反思与核查，不是直接把辩题丢给模型。



这个回合循环让系统具备“可被追问”的智能：如果发言可信，能够说明依据；如果有风险，能够在发布前或评分时被发现。

—— 创新点之一

AI 辩论最危险的地方，是“听起来很有道理”。

所以系统不把流畅文本当作最终答案，而是把事实来源、引用编号、核查结果和裁判扣分放进同一条链路。目标不是让模型永不出错，而是让错误在训练流程里被发现、被标记、被复盘。

资料入库	用户上传材料形成可检索来源，发言引用绑定为 `[kb-x]` 这类可追踪编号。
发言前	RAG 根据当前环节、持方和历史战场检索资料，提示词约束不得编造来源。
发言后	事实核查节点检查风险；不通过时回到生成节点重写，避免问题直接进入公开发言。
发布前	`sanitize_citations` 移除未入库引用，让伪造编号不能伪装成真实资料。
赛后	裁判报告记录主战场、失误与幻觉风险，训练者可以据此复盘证据使用质量。

—— 创新点之二

系统刻意保留学生的发言权，让 AI 做陪练、资料官和教练。

在人机模式中，轮到用户时自动暂停。学生可以查看当前战场、资料引用和 AI 教练建议，也可以让系统代拟草稿，但最终提交仍由学生决定。

这让项目从“AI 替我完成比赛”转向“AI 帮我练习判断”。系统还会审核低信息量或不合适发言，并可选择启用超时扣分、自信度摄像头训练、语音输入与 TTS 朗读。

三种训练视角

上下文模式：适合学习，能看到更多推理和队内准备。

真实辩论模式：适合实战，只看到自己应当看到的信息。

上帝视角：适合老师和复盘，观察双方策略如何变化。

人机协同的边界越清楚，训练价值越高：AI 给方向和证据，人完成判断与表达。

—— 跨端运行

同一套系统覆盖电脑主持、手机加入、桌面发行和弱环境演示。

电脑端

浏览器访问 `127.0.0.1:5173`，后端默认 `9000`。主持人可创建房间、管理赛程、观察进度，并在管理页查看存储与运行状态。

手机端

响应式 Web 布局支持同 WiFi 加入，宾客端使用竖屏样式。联机席位、在线状态和发言同步由 WebSocket 维护。

发行版

Electron 桌面壳复用 Web UI；便携 Python/Node 与启动脚本支持 U 盘部署。MongoDB/Redis 未启动时可内存降级，先保证演示链路跑通。

这部分创新来自现实约束：真正的学生项目不只要“能开发”，还要在比赛现场、学校电脑和同学手机上能打开。

—— 工程结果

项目不只完成演示界面，也补齐了测试、导出、回放和运维入口。

代码结构	后端按 API、服务、工作流、数据库、配置拆分；前端按页面、辩论室特性组件和通用 hooks 拆分。
实时能力	WebSocket 事件覆盖 snapshot、speech_chunk、awaiting_user、debate_stepped、debate_finished 等关键状态。
质量保障	测试覆盖引用校验、消息可见性、工作流推进、WebSocket、联机流程、隧道与用户发言评审等模块。
可观测性	管理页提供房间列表、单房间诊断、LLM 用量粗统计、自动推进停止与恢复等运维操作。
输出沉淀	全场发言可以导出 Markdown/PDF；回放页和分享链接让一次训练

—— 应用价值

把 AI 素养训练落到一次具体、可复盘的辩论里。

学生在使用系统时，会自然接触 workflow、提示词、RAG、输出校验、多智能体协同和实时通信。这些技术不再停留在概念层，而是在一场辩论中变成可观察、可解释的系统行为。

面向四类场景

课堂：老师用 AI 自主辩论演示“如何建立论证与反驳”。

社团：同学加入正反方，AI 补位，降低组局成本。

竞赛：学生讲清楚从需求分析到系统实现的完整过程。

个人训练：导出逐字稿与裁判报告，追踪自己的表达改进。

最终目标不是证明 AI 比人会辩论，而是让人更会提出问题、核验证据、组织表达。